AI 101 for Mechanical Engineers · Module 14

Tools act; agents coordinate

Lesson 14.3

What you will learn

> Distinguish an assistant, a tool, an API and an agent, and separate code generation, execution and engineering verification.

Lesson overview

Module 14 · Lesson 3

Tools act; agents coordinate

AI 101 for Mechanical Engineers

1 Four roles (course working definitions)

Assistant
An interface that helps a user formulate or inspect work.

Tool
Performs a defined operation, e.g. runs Python or searches local documents.

API
The software interface through which a program requests an operation.

Agent
Selects and coordinates actions over several steps toward a goal, with defined autonomy.

Example: ReAct alternates reasoning steps with actions such as Wikipedia queries. Not the same as the reinforcement-learning agent of Lesson 4.3.

A responsible person sets permissions, input limits and approval rules, and verifies the final output.

2 Draft → run → verify → approve

Draft code → Run with a tool → Verify → Approve action

Running without an error is not proof of an engineering-correct result.

Before a real-system action, check:

- ✓ Syntax and tests
- ✓ Equations and units
- ✓ Input limits and operating domain
- ✓ Permissions and logs
- ✓ Independent numerical check and approval by the responsible person

3 Autonomy needs controls

Assistant suggests
person reviews the output

Tool runs an operation
tests, input limits, logs

Agent acts over steps
permissions, limits, logs, escalation path, approval

More autonomy needs more explicit controls. Responsibility stays with people, not the software.

The course prompt builder runs locally: it sends no data and trains no model.

Key takeaway

Generation, execution and verification are separate stages.
Software can act, but a responsible person checks and approves real-system actions.

AI 101 for Mechanical Engineers · Lesson 14.3

© 2026 Prof. Mazharul Islam · CC BY 4.0

Each component has a different role

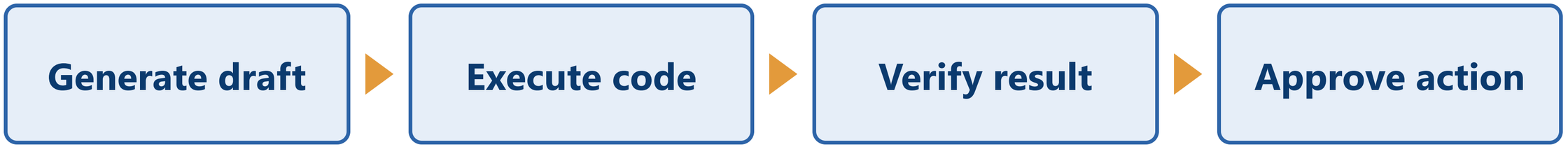
Component	Working definition
Assistant	Interface that supports a user’s task
Tool	Performs a defined operation, e.g. executing Python
API	Interface through which software requests an operation
Agent system	Can select and coordinate actions across several steps

> An agent works toward a stated goal with a defined degree of autonomy.

People set the limits for agents

- ReAct is one research example: reasoning alternates with actions [94]
- Example actions: queries to a Wikipedia interface [94]
- A responsible person defines permissions, input limits and approvals [9]
- That person verifies the final engineering output [9]

Drafting, execution and correctness are separate



- Generated code is a draft [78]
- Running without a software error does not prove engineering correctness

Checks before a tool acts on a real system

1. Syntax and tests
2. Equations and units
3. Input limits, operating domain and permissions
4. Logs, and independent verification of numerical results
5. Approval from the responsible person

The course prompt builder works locally

It does

- Build text for review
- Allow optional use elsewhere

It does not

- Call an external language-model service
- Send course data or train a model

> A local check cannot establish that a real machine, equation or operating decision is safe.

Two different meanings of “agent”

Agent in this lesson

- Coordinates bounded actions across steps

Reinforcement-learning agent (Lesson 4.3)

- Learns a policy from reward feedback

More autonomy increases the need for explicit controls; it does not transfer responsibility to software.

Document every role and its controls

- A tool carries out an operation; an API defines the request
- An assistant helps a user formulate or inspect work
- Document permissions, limits, logs and an escalation path

CHECK YOUR UNDERSTANDING

Generated code runs without errors. Is it ready to support a real-system action?

Answer before moving on.

CHECK YOUR UNDERSTANDING

Generated code runs without errors. Is it ready to support a real-system action?

> No. Running without errors does not show correctness. It still needs numerical, unit and physics checks, independent verification, and approval from the responsible person.

Key takeaways

- Assistants, tools, APIs and agents have different roles
- Keep code generation, execution and verification as separate stages
- Successful execution does not prove engineering correctness
- Define permissions, logs, operating limits and human approval first

References

[9] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Gaithersburg, MD, USA, 2023. doi: 10.6028/NIST.AI.100-1

[78] M. Chen et al., “Evaluating large language models trained on code,” 2021, arXiv:2107.03374. doi: 10.48550/arXiv.2107.03374

[94] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. 11th Int. Conf. Learn. Represent. (ICLR), 2023.

Numbers match the course reference list. The written lesson is the authoritative text.